

Computer and Network Security — Lecture 08

Dr. Karim Lounis

Jan - Jun 2026



Asymmetric Cryptography (Con't)

Asymmetric Cryptography

RSA public-key Encryption Scheme

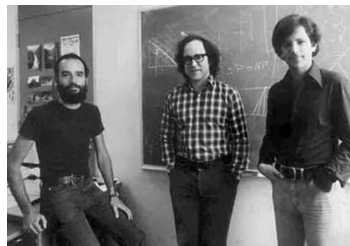
RSA

Definition

RSA. Invented in 1977. It is an asymmetric cryptosystem that relies on the difficulty of factorizing numbers into two large prime numbers.

It has the following features:

- RSA is one of the most widely used public-key cryptosystems due to its security, efficiency, and versatility.
- Used to generate digital signatures and key establishment in various applications (e.g., ePassport, TLS, EMV cards, etc).
- Keys have various lengths, minimum 2048 bits, up to 4096 bits as per NIST's recommendations (based on the level of sensitivity of the protected information).



Shamir, Rivest, and Adleman

Future quantum computers might pose a significant threat to RSA by efficiently factoring large numbers.

RSA Encryption and Decryption

Keys generation (done once): RSA starts by generating a pair of keys for a given user as follows (common notation from literature):

- ① Randomly choose two distinct large prime numbers p and q in $\mathbb{Z}$.
- ② Compute the modulus $n = p \times q$.
- ③ Compute the Euler's totient function $\phi(n)$:

$$\phi(n) = \phi(p \times q) = \phi(p) \times \phi(q) = (p - 1) \times (q - 1)$$

- ④ Public key (e, n) : Randomly choose a number $e \in \mathbb{Z}$ s.t.

$$3 < e < \phi(n) \text{ and } \gcd(e, \phi(n)) = 1$$

- ⑤ Private key d : Compute $d \in \mathbb{Z}_{\phi(n)}^+$ s.t $d \equiv e^{-1}[\phi(n)]$.

Encryption: Having a plaintext X and the public key $e \in \mathbb{Z}$ the ciphertext is computed as follows: $Y = \mathcal{E}_e(X) = X^e[n]$.

Decryption: Having a ciphertext Y and the private key $d \in \mathbb{Z}_{\phi(n)}^+$ the plaintext is retrieved as follows: $X = \mathcal{D}_d(Y) = Y^d[n]$.

RSA Encryption and Decryption

Keys generation (done once): RSA starts by generating a pair of keys for a given user as follows (common notation from literature):

- ① Randomly choose two distinct large prime numbers p and q in $\mathbb{Z}$.

② **Example:** Consider the following:

- ③ ① Bob chooses $p = 3$ and $q = 11$. Hence, $n = 33$ and $\phi(n) = 20$. He chooses $e = 3$ and computes the private key $d \equiv e^{-1}[\phi(n)] = 7$.

- ④ ② Bob sends his public key (e, n) to Alice.

- ⑤ ③ Alice, takes the secret message $x = 4$, encrypts it using Bob's public key as follows: $y = \mathcal{E}_3(x) = x^e = 4^3[33]$, and sends the ciphertext $y = 31$ to Bob.

- En
pho
- ④ ④ Bob receives the message and performs decryption as follows: $x = \mathcal{D}_7(y) = (x^e)^d = x^1 \equiv 31^7[33] \equiv (-2)^7 \equiv -128[33] \equiv 4$

Decryption: Having a ciphertext Y and the private key $d \in \mathbb{Z}$ the plaintext is retrieved as follows: $X = \mathcal{D}_d(Y) = Y^d[n]$.

RSA Encryption and Decryption

Keys generation (done once): RSA starts by generating a pair of keys for a given user as follows (common notation from literature):

- ① Randomly choose two distinct large prime numbers p and q in $\mathbb{Z}$.
- ② **Exercise (S1):** Zoubir want to set up RSA cipher and use it to send and receive encrypted messages. He selects the modulus $n=39$ and randomly choosed the value $e = 7$ as his public key. What is his private key d ?
- ③

$$\varphi(n) = \varphi(p \times q) = \varphi(p) \times \varphi(q) = (p-1) \times (q-1)$$

- ④ Public key (e, n) : Randomly choose a number $e \in \mathbb{Z}$ s.t.
 $1 < e < \phi(n)$ and $\gcd(e, \phi(n)) = 1$
- ⑤ Private key d : Compute $d \in \mathbb{Z}_{\phi(n)}^+$ s.t $d \equiv e^{-1}[\phi(n)]$. (the multiplicative inverse of e modulo $\phi(n)$)

Encryption: Having a plaintext X and the public key $e \in \mathbb{Z}$ the ciphertext is computed as follows: $Y = \mathcal{E}_e(X) = X^e[n]$.

Decryption: Having a ciphertext Y and the private key $d \in \mathbb{Z}$ the plaintext is retrieved as follows: $X = \mathcal{D}_d(Y) = Y^d[n]$.

RSA Encryption and Decryption

Keys generation (done once): RSA starts by generating a pair of keys for a given user as follows (common notation from literature):

- ① Randomly choose two distinct large prime numbers p and q in $\mathbb{Z}$.
- ② **Exercise (S1):** Zoubir want to set up RSA cipher and use it to send and
- ③ receive encrypted messages. He selects the modulus $n=39$ and randomly choosed the value $e = 7$ as his public key. What is his private key d ?

- Solution.** The private key is $d = 7$. To compute it, you first need to
- ④ factorize the modulus into two prime numbers: $n = 39 = 3 \cdot 13$. You compute the Euler's totient function over n : $\phi(39) = \phi(3) \cdot \phi(13) = 24$.
 - ⑤ The private key of Zoubir is the multiplicative inverse of 7 modulo 24.

multiplicative inverse of e modulo $\phi(n)$)

Encryption: Having a plaintext X and the public key $e \in \mathbb{Z}$ the ciphertext is computed as follows: $Y = \mathcal{E}_e(X) = X^e[n]$.

Decryption: Having a ciphertext Y and the private key $d \in \mathbb{Z}$ the plaintext is retrieved as follows: $X = \mathcal{D}_d(Y) = Y^d[n]$.

RSA Encryption and Decryption

Keys generation (done once): RSA starts by generating a pair of keys for a given user as follows (common notation from literature):

- ① Randomly choose two distinct large prime numbers p and q in $\mathbb{Z}$.
- ② **Exercise (S2):** Zoubir want to set up RSA cipher and use it to send and receive encrypted messages. He selects the modulus $n=91$ and randomly choosed the value $e = 5$ as his public key. What is his private key d ?
- ③

$$\varphi(n) = \varphi(p \times q) = \varphi(p) \times \varphi(q) = (p-1) \times (q-1)$$

- ④ Public key (e, n) : Randomly choose a number $e \in \mathbb{Z}$ s.t.
 $1 < e < \phi(n)$ and $\gcd(e, \phi(n)) = 1$
- ⑤ Private key d : Compute $d \in \mathbb{Z}_{\phi(n)}^+$ s.t $d \equiv e^{-1}[\phi(n)]$. (the multiplicative inverse of e modulo $\phi(n)$)

Encryption: Having a plaintext X and the public key $e \in \mathbb{Z}$ the ciphertext is computed as follows: $Y = \mathcal{E}_e(X) = X^e[n]$.

Decryption: Having a ciphertext Y and the private key $d \in \mathbb{Z}$ the plaintext is retrieved as follows: $X = \mathcal{D}_d(Y) = Y^d[n]$.

RSA Encryption and Decryption

Keys generation (done once): RSA starts by generating a pair of keys for a given user as follows (common notation from literature):

- ① Randomly choose two distinct large prime numbers p and q in $\mathbb{Z}$.
- ② **Exercise (S2):** Zoubir want to set up RSA cipher and use it to send and
- ③ receive encrypted messages. He selects the modulus $n=91$ and randomly choosed the value $e = 5$ as his public key. What is his private key d ?

- Solution.** The private key is $d = 21$. To compute it, you first need to
- ④ factorize the modulus into two prime numbers: $n = 91 = 7 \cdot 13$. You compute the Euler's totient function over n : $\phi(91) = \phi(7) \cdot \phi(13) = 72$.
 - ⑤ The private key of Zoubir is the multiplicative inverse of 5 modulo 72.

multiplicative inverse of e modulo $\phi(n)$)

Encryption: Having a plaintext X and the public key $e \in \mathbb{Z}$ the ciphertext is computed as follows: $Y = \mathcal{E}_e(X) = X^e[n]$.

Decryption: Having a ciphertext Y and the private key $d \in \mathbb{Z}$ the plaintext is retrieved as follows: $X = \mathcal{D}_d(Y) = Y^d[n]$.

Fast Exponentiation

What is the time complexity of computing the exponentiation?

- Naive:

We just keep multiplying ...

- Fast multiplication:

We just keep multiplying, but in a smarter way, ...

Let us consider the example of x^4

- Naive:

You do $x \times x = x^2$ **then** $x^2 \times x = x^3$ **then** $x^3 \times x = x^4$

- Fast multiplication:

You do $x \times x = x^2$ **then** $x^2 \times x^2 = x^4$

Fast Exponentiation

What is the time complexity of computing the exponentiation?

- Naive:

We just keep multiplying ...

- Fast multiplication:

We just keep multiplying, but in a smarter way, ...

Let us consider the example of x^4

- Naive:

You do $x \times x = x^2$ **then** $x^2 \times x = x^3$ **then** $x^3 \times x = x^4$

- Fast multiplication:

You do $x \times x = x^2$ **then** $x^2 \times x^2 = x^4$

So, 3 operations in Naive approach, and 2 in the fast multiplication.

Fast Exponentiation

What is the time complexity of computing the exponentiation?

- Naive:

We just keep multiplying ...

- Fast multiplication:

We just keep multiplying, but in a smarter way, ...

Let us consider another example of x^8

- Naive:

You do $x \times x = x^2$ **then** $x^2 \times x = x^3$ **then ... then** $x^7 \times x = x^8$

- Fast multiplication:

You do $x \times x = x^2$ **then** $x^2 \times x^2 = x^4$ **then** $x^4 \times x^4 = x^8$

Fast Exponentiation

What is the time complexity of computing the exponentiation?

- Naive:

We just keep multiplying ...

- Fast multiplication:

We just keep multiplying, but in a smarter way, ...

Let us consider another example of x^8

- Naive:

You do $x \times x = x^2$ **then** $x^2 \times x = x^3$ **then ... then** $x^7 \times x = x^8$

- Fast multiplication:

You do $x \times x = x^2$ **then** $x^2 \times x^2 = x^4$ **then** $x^4 \times x^4 = x^8$

So, 7 operations in Naive approach, and 3 in the fast multiplication.

Fast Exponentiation

What is the time complexity of computing the exponentiation?

- Naive:

We just keep multiplying ...

- Fast multiplication:

We just keep multiplying, but in a smarter way, ...

Let us consider one last example, where we want to compute $x^{2^{1024}}$

- Naive:

You do $x \times x = x^2$ **then** $x^2 \times x = x^3$ **then ... then**
 $x^{2^{1024}-1} \times x = x^{2^{1024}}$

- Fast multiplication:

You do $x \times x = x^2$ **then** $x^2 \times x^2 = x^4$ **then** $x^{2^{1023}} \times x^{2^{1023}} = x^{2^{1024}}$

Fast Exponentiation

What is the time complexity of computing the exponentiation?

- Naive:

We just keep multiplying ...

- Fast multiplication:

We just keep multiplying, but in a smarter way, ...

Let us consider one last example, where we want to compute $x^{2^{1024}}$

- Naive:

You do $x \times x = x^2$ **then** $x^2 \times x = x^3$ **then ... then**
 $x^{2^{1024}-1} \times x = x^{2^{1024}}$

- Fast multiplication:

You do $x \times x = x^2$ **then** $x^2 \times x^2 = x^4$ **then** $x^{2^{1023}} \times x^{2^{1023}} = x^{2^{1024}}$

So, $2^{1024} - 1$ operations in Naive approach, and 1024 in the FM.

Fast Exponentiation

What is the time complexity of computing the exponentiation?

- Naive:

But, what if the exponent is not a power of 2? which is mostly the case in cryptography.

Let us consider x^{26}

Let You do $x \times x = x^2$ **then** $x^2 \times x = x^3$ **then** $x^3 \times x^3 = x^6$ **then**
 $x^6 \times x^6 = x^{12}$ **then** $x^{12} \times x = x^{13}$ **then** $x^{13} \times x^{13} = x^{26}$
 So 6 multiplications

What is the pattern?

You do $x \times x = x^2$ **then** $x^2 \times x^2 = x^4$ **then** $x^4 \times x^4 = x^8$ **then** $x^8 \times x^8 = x^{16}$ **then** $x^{16} \times x^{10} = x^{26}$

So, $2^{1024} - 1$ operations in Naive approach, and 1024 in the FM.

Fast Exponentiation (Square-and-multiply algorithm)

Definition

Square-and-multiply algorithm. Is a fast method for computing exponentiation using the binary representation of the exponent. It relies on squaring and multiplying with the base following a pattern.

We would use the Square-and-multiply algorithm (a.k.a., binary method, or left-to-right exponentiation):

What is the pattern? **The binary representation**

E.g., $26 = (11010)_2$. We want to compute $x^{26} = x^{11010_2}$. We need to reconstruct the binary sequence starting from $x = 1$.

- Scan the binary representation from left to right.
- In every iteration we square (shift to left, i.e., add a 0 to the right).
- If current bit is one, multiply by x (add one to the sequence).

RSA Cryptanalysis

If Oscar want to break RSA, he will have e and n , and apply the EEA. But he is missing one thing, which is: ...?

Cryptanalysis. There have been several attempts to break RSA:

- ① Breaking RSA consists of finding out p and q knowing n . The temporal complexity is exponential w.r.t. the size of the key (the length of n).

$p \in \mathbb{Z}$ and $q \in \mathbb{Z}$ have to be large enough

- ② Cyclic attack (Fermat's little theorem attack), where the attacker keeps encrypting the ciphertext for a number of times till hitting the plaintext, i.e. $(\dots((m^e)^e)^e\dots)^e = m^{e^i} = m^e[n]$, may succeed, since $m = m^{e^{i-1}}[n]$.

$e \in \mathbb{Z}$, has to be large for expensive computation

- ③ Side-channel attack. These attacks exploit unintended information leakage from the implementation of the RSA algorithm, such as timing differences (and power consumption) during encryption or decryption operations.

Introduce randomization

RSA Cryptanalysis

If Oscar want to break RSA, he will have e and n , and apply the EEA. But he is missing one thing, which is: $\phi(n)$, the Euler's totient on n

Cryptanalysis. There have been several attempts to break RSA:

- ① Breaking RSA consists of finding out p and q knowing n . The temporal complexity is exponential w.r.t. the size of the key (the length of n).

$p \in \mathbb{Z}$ and $q \in \mathbb{Z}$ have to be large enough

- ② Cyclic attack (Fermat's little theorem attack), where the attacker keeps encrypting the ciphertext for a number of times till hitting the plaintext, i.e. $(\dots((m^e)^e)^e\dots)^e = m^{e^i} = m^e[n]$, may succeed, since $m = m^{e^{i-1}}[n]$.

$e \in \mathbb{Z}$, has to be large for expensive computation

- ③ Side-channel attack. These attacks exploit unintended information leakage from the implementation of the RSA algorithm, such as timing differences (and power consumption) during encryption or decryption operations.

Introduce randomization

RSA Cryptanalysis

If Oscar want to break RSA, he will have e and n , and apply the EEA. But he is missing one thing, which is: **the factorization of n**

Cryptanalysis. There have been several attempts to break RSA:

- ① Breaking RSA consists of finding out p and q knowing n . The temporal complexity is exponential w.r.t. the size of the key (the length of n).

$p \in \mathbb{Z}$ and $q \in \mathbb{Z}$ have to be large enough

- ② Cyclic attack (Fermat's little theorem attack), where the attacker keeps encrypting the ciphertext for a number of times till hitting the plaintext, i.e. $(\dots((m^e)^e)^e\dots)^e = m^{e^i} = m^e[n]$, may succeed, since $m = m^{e^{i-1}}[n]$.

$e \in \mathbb{Z}$, has to be large for expensive computation

- ③ Side-channel attack. These attacks exploit unintended information leakage from the implementation of the RSA algorithm, such as timing differences (and power consumption) during encryption or decryption operations.

Introduce randomization

ElGamal public-key Encryption Scheme

ElGamal

Definition

ElGamal. Invented in 1985 by Taher Elgamal (a.k.a., the Father of SSL). It is an asymmetric cryptosystem that relies on the difficulty of solving the discrete logarithm problem.

It has the following features:

- The cryptosystem includes both an encryption-decryption scheme, as well as a digital signature generation and verification scheme.
- ElGamal scheme can be viewed as Diffie-Helman key agreement in key transfer mode.
- ElGamal cryptosystem is usually used as part of a hybrid cryptosystem.



ElGamal's signature scheme became the basis for Digital Signature Algorithm (DSA) adopted by NIST as the Digital Signature Standard (DSS).

ElGamal

Keys generation (done once): It starts by generating a pair of keys for a given user (Alice) as follows (common notation from literature):

- ① Generate a description of a finite cyclic multiplicative group $\mathcal{G}$ of order $q \in \mathbb{Z}$ (large prime number) with generator $g \in \mathbb{Z}$ (note that the same cyclic group can be used multiple times).
- ② Randomly choose an integer x from $\{1, \dots, q-1\}$.
- ③ Compute $h = g^x [q]$.
- ④ The public key is (q, g, h) .
- ⑤ Alice publishes this tuple as her public key and retains the value $x \in \mathbb{Z}_q^*$ as her private key.

$$\text{Pair of key} = \begin{cases} x \in \mathbb{Z}_q^* & \text{Private key,} \\ (q, g, h) & \text{Public key} \end{cases}$$

ElGamal

Encryption: Having a plaintext M , Bob uses Alice's public key to encrypt the message as follows:

- ① Map the message M to an element of $m \in \mathcal{G}$ (mapping function).
- ② Randomly choose an integer y from $\{1, \dots, q-1\}$ (ephemeral key).
- ③ Compute $s = h^y [q]$ (shared secret).
- ④ Compute $c_1 = g^y [q]$
- ⑤ Compute $c_2 = m \cdot s [q]$
- ⑥ Bob, sends the ciphertext (c_1, c_2) to Alice (ciphertext is $2 \times$ larger)

Decryption: Upon receiving the ciphertext (c_1, c_2) , Alice:

- ① Computes $s = c_1^x [q]$ (to obtain the shared secret).
- ② Computes the inverse $s^{-1} [q]$ in the group cyclic $\mathcal{G}$.
- ③ Computes $m = c_2 \cdot s^{-1} [q]$, then map m back to M .

Exercise: To send encrypted and “authenticated” data, Alice decides to use ELGamal cryptosystem. To that end, Bob chooses a prime number $q = 2357$ and a generator $g = 2$ of the multiplicative group $\mathbb{Z}_{2357}^*$. He randomly chooses his private key as $x = 17$.

- ① Compute the public key of Bob that will be sent to Alice.
- ② Alice wants to send the message “OK” to Bob. It uses a direct mapping function (e.g., A=1, B=2, ..., Z=26). Encode the message into a numerical value.
- ③ Encrypt the message to be sent to Bob (put $y = 1520$).
- ④ Being Bob, decrypt the received message.

Exercise: To send encrypted and “authenticated” data, Alice decides to use ElGamal cryptosystem. To that end, Bob chooses a prime number $q = 2357$ and a generator $g = 2$ of the multiplicative group $\mathbb{Z}_{2357} - \{0\}$. He randomly chooses his private key as $x = 17$.

- ① We have $h = g^x[q] = 2^{17}[2357] = 1437$, Bob’s public key is the tuple $(2357, 2, 1437)$.
- ② The message will be encoded into 15.11, i.e., 1511 in $\mathcal{G}$ (you could also choose to encode it as 15 and 11).
- ③ Let’s encrypt 1511. Alice, chooses $y = 1520$, and computes:

$$c_1 = 2^{1520}[2357] = 1430 \text{ and } c_2 = (1511) \cdot 1437^{1520}[2357] = 1401$$

Alice then sends $(c_1, c_2) = (1430, 1401)$ to Bob.

- ④ Bob receives the ciphertext $(1430, 1401)$ and computes the encoded message: $m = c_2 \cdot ((c_1^x)^{-1}[2357]) = 1401 \cdot (1564)[2357] = 1511$.

ElGamal doesn't inherently handle message formatting or structure. Also, it is computationally expensive compared to symmetric schemes. It is primarily used for encrypting hashes (to generate digital signatures) and encrypting sessions keys (key establishment).

Merkle-Hellman public-key Encryption Scheme

Merkle-Hellman cryptosystem

Definition

Merkle-Hellman Knapsack scheme. Invented in 1978 by Ralph Merkle and Martin Hellman, was one of the first public-key cryptosystems. It relies on the difficulty of solving an instance of the subset sum problem, known as the *superincreasing subset sum problem*.

It has the following features:

- The public key consists of a list of positive integers (the “knapsack”) — a non-super increasing list of positive integers, whereas the private key is a superincreasing list of numbers related to the one of the knapsack.
- No longer considered secure for practical use. It was shown to be vulnerable (by A. Shamir) to cryptanalysis attacks in the early 1980s, that allow attackers to recover the private key from the public key with polynomial-time complexity.



Superincreasing Subset Sum Problem

Definition

Superincreasing Subset Sum Problem is an instance of the mathematical subset sum problem. This problem states the following:

Let $\mathcal{S}$ be a superincreasing sequence $(b_1, b_2, \dots, b_n)$ of positive integers, such that $\forall b_i > \sum_{j=1}^{i-1} b_j$ for each $i, 2 \leq i \leq n$.

Let s be an integer, which is the sum of a subset of the b_i .

Find a selection tuple $(x_1, x_2, \dots, x_n)$ where $x_i \in \{0, 1\}$, s.t. $\sum_{i=1}^n x_i \cdot b_i = s$.

Applying the Glouton algorithm, we can solve the problem.

Example. Consider the following sequence $\mathcal{S} = \{1, 3, 6, 13, 27, 52\}$ and let the integer $s = 30$ be the target sum:

The subset is: $\mathcal{S}' = \{3, 27\}$. This is because:

- $3 + 27 = 30$ (satisfies the target sum)
- $3 > 1$ and $27 > (1 + \dots + 13) = 23$ (satisfies the superincreasing constraint)

Superincreasing Subset Sum Problem

Definition

Superincreasing Subset Sum Problem is an instance of the mathematical subset sum problem. This problem states the following:

Let $\mathcal{S}$ be a superincreasing sequence $(b_1, b_2, \dots, b_n)$ of positive integers, such that $\forall b_i > \sum_{j=1}^{i-1} b_j$ for each $i, 2 \leq i \leq n$.

Let s be an integer, which is the sum of a subset of the b_i .

Find a tuple $(x_1, x_2, \dots, x_n)$ where $x_i \in \{0, 1\}$, s.t. $\sum_{i=1}^n x_i \cdot b_i = s$

Applying the Glouton “greedy” algorithm, we can solve the problem.

However, if the sequence is not superincreasing, then finding the tuple $(x_1, x_2, \dots, x_n)$ would be to try all possible cases (i.e., all $b_i, i \in \{1..n\}$). If the sequence is large enough, the algorithm becomes uncomputable.

Merkle-Hellman cryptosystem

Key generation. It goes as follows:

- ① Choose a block size n to encrypt integers up to n bits.
- ② Choose a random superincreasing sequence $\mathcal{S}$ of n positive integers.
- ③ Choose a random integer q s.t. $q > \sum_{i=1}^n s_i, s_i \in \mathcal{S}$.
- ④ Choose a random integer r s.t. $\gcd(r, q)=1$.
- ⑤ Calculate the sequence $\mathcal{T}$, s.t. $\forall t_i \in \mathcal{T}, t_i = s_i \cdot r[q]$.
- ⑥ The public key is $\mathcal{T}$, and the private key is $(\mathcal{S}, q, r)$.

Encryption: To encode a message $m = (m_1 m_2 \dots m_n)$ of n bits. Select each $t_i \in \mathcal{T}$ for which m_i is non-zero, and add them together to generate the ciphertext as follows: $c = \sum_{i=1}^n m_i \cdot t_i$.

Merkle-Hellman cryptosystem

Decryption: To decrypt the ciphertext c , we must find the subset of $\mathcal{T}$ which sums to c . We try to find a subset of $\mathcal{S}$ that can be solved in polynomial time.

- ① Compute r^{-1} the inverse of r modulo q .
- ② Calculate $c' = c \cdot r^{-1}[q]$.
- ③ Solve the subset sum problem for c' using $\mathcal{S}$, to find X where:

$$X = (x_1, x_2, \dots, x_k), \text{ s.t., } c' = \sum_{i=1}^k s_{x_i}.$$

- ④ Construct the message m with a 1 in each x_i bit position and a 0 in all other bit positions:

$$m = \sum_{i=1}^k 2^{n-i} * x_i$$

Merkle-Hellman cryptosystem

Exercise: Consider the following superincreasing sequence:

$$\mathcal{S} = \{2, 7, 11, 21, 42, 89, 180, 354\}$$

The sum of this sequence is 706. Consider a value for q to be 881. Consider the value of r to be 558, s.t., $\gcd(558, 881)=1$.

- ① Construct the public key $\mathcal{T}$.
- ② Encrypt the letter “a” (note that ascii code of “a” is 97).
- ③ Decrypt the ciphertext back to “a”.

Merkle-Hellman cryptosystem

Exercise: we have:

- ① Construct the public key $\mathcal{T}$.

$$\mathcal{T} = \{235, 382, 852, 265, 530, 326, 6, 188\}$$

- ② Encrypt the letter “a” (note that ascii code of “a” is 97).

$$c = (235 \times 0) + (382 \times 1) + \dots + (188 \times 1) = 1422$$

- ③ Decrypt the ciphertext back to “a”.

- The inverse of $r = 558$ [881] is 30 (i.e., $558 \cdot 30 \equiv 1$ [881]).
- Compute $c' = c \cdot r^{-1}[q] = 1422 \cdot 30$ [881] = 372
- Use the greedy algo to decompose 372 into a sum of s_i .
- Find that $372 = 354 + 11 + 7 = s_8 + s_3 + s_2$, so $X = (2, 3, 8)$.
- Message is: $m = \sum_{i=1}^3 2^{n-x_i} = 1 + 32 + 64 = 97$.
- Retrieve the plaintext: $\text{Asci}^{-1}(97) = \text{“a”}$.

Message Digestion — Hashing Algorithms

Message Digestion

Definition

Message Digestion. A.k.a., Hashing. It is the process of transforming a message into a unique and compact representation using a cryptographic function (one-way function). The output of the function is called a digital fingerprint, message digest, or simply hash value.

It has the following features:

- The hash function takes an arbitrary message of any length and produces a fixed-size string of characters (hash).
- This hash value is highly sensitive to any changes in the original message. Even a single bit change in the message results in a completely different hash value.

```
(base) mirak@Karims-MacBook-Pro ~ % md5 -s ensia
MD5 ("ensia") = 6e68c9e9b9f00e859e2726da2313f30d
(base) mirak@Karims-MacBook-Pro ~ % md5 -s Ensia
MD5 ("Ensia") = b6a9453aeaa53565c65fd44fd631d196
(base) mirak@Karims-MacBook-Pro ~ % md5 -s ensiA
MD5 ("ensiA") = d605ba782a19c213ba3c61b071d89ff7
(base) mirak@Karims-MacBook-Pro ~ % md5 -s eNsia
MD5 ("eNsia") = 8916b801641dbba2aa4a55765b3ff80d
(base) mirak@Karims-MacBook-Pro ~ % md5 -s ENSIA
MD5 ("ENSIA") = ae4ec965bf1c48169560a55d2a3d1a44
(base) mirak@Karims-MacBook-Pro ~ % md5 -s ensiA
MD5 ("ensiA") = 9f4460973a2c23cb916970d9cc875874
(base) mirak@Karims-MacBook-Pro ~ % md5 -s eNsia
MD5 ("eNsia") = 8916b801641dbba2aa4a55765b3ff80d
(base) mirak@Karims-MacBook-Pro ~ % md5 -s "ensia "
MD5 ("ensia ") = 8a92957275b72ee2a6967db8ff5c93f3
```

MD5 hash function over “ensia”

It is **mainly** used to generate digital signatures, data integrity verification (at rest and in transit), and password storage (e.g., `/etc/shadow`).

Message Digestion

Good hash function has the following properties:

- **Collision resistance.** It is extremely hard, if not impossible, to find two different messages that generate the same hash value:

$$\forall m, m' \in \mathcal{M} \mid m \neq m' : \mathcal{H}(m) \neq \mathcal{H}(m')$$

- **Preimage resistance.** Given a hash value $\mathcal{H}(m)$, it is mathematically infeasible to find the original message $m \in \mathcal{M}$. I.e., $\nexists \mathcal{H}(m)^{-1}$.
- **Second preimage resistance.** Given a message $m \in \mathcal{M}$ and its hash value $\mathcal{H}(m)$, it is infeasible to find another message $m' \in \mathcal{M}$ that produces the same hash $\mathcal{H}(m)$. I.e., $\forall m \in \mathcal{M}, \nexists m' \in \mathcal{M} : \mathcal{H}(m) = \mathcal{H}(m')$.

Popular message digest algorithms (functions) are: MD5 (Message Digest 5), SHA-1 (Secure Hash Algorithm 1), SHA-256, SHA-384, SHA-512, BLAKE2, BLAKE3, Whirlpool, Tiger, MD4, ... FarmHash, CityHash, and bcrypt.

```
(rodus@kali)-[~]
$ sudo cat /etc/shadow | grep "target"
target:$y$j9T$3IKHjT8PEsTbd3r64kKuo/$2zT.m7apk9Yw27WN2bTCDyMyTOLYV4TizC9jd3mOZ17:19739:0:99999:7:::
```

The prefix `$y$` means `bcrypt`, the value `j9T` is the salt, & the rest is the hash.

Type of Message Digests

When hash function are used to detect whether the message input has been altered (at rest or in transit), the hash value is called a modification detection code (**MDC**), or message integrity code (**MIC**).

When hash function involves the use of a shared key to provide data origin authentication as well as data integrity, the hash value is called hashed message authentication code (**HMAC**).

HMAC is a specific type of **MAC** — message authentication code — that uses a cryptographic hash function along with a secret key.

Other MAC can be generated using a secret key and a symmetric block cipher (instead of a hash function). E.g., use a block cipher (like AES) in CBC-MAC (Cipher Block Chaining mode). [[used in Wi-Fi WPA1/2/3](#)]

Comparison of SHA functions (cf., [source](#))

Algorithm and variant		Output size (bits)	Internal state size (bits)	Block size (bits)	Rounds	Operations	Security against collision attacks (bits)	Security against length extension attacks (bits)	Performance on Skylake (median cpb) ^[59]		First published
									Long messages	8 bytes	
MD5 (as reference)		128	128 (4 × 32)	512	4 (16 operations in each round)	And, Xor, Or, Rot, Add (mod 2 ³²)	≤ 18 (collisions found) ^[60]	0	4.99	55.00	1992
SHA-0		160	160 (5 × 32)	512	80	And, Xor, Or, Rot, Add (mod 2 ³²)	< 34 (collisions found)	0	≈ SHA-1	≈ SHA-1	1993
SHA-1							< 63 (collisions found) ^[61]		3.47	52.00	1995
SHA-2	SHA-224	224	256 (8 × 32)	512	64	And, Xor, Or, Rot, Shr, Add (mod 2 ³²)	112	32	7.62	84.50	2004
	SHA-256	256					128	0	7.63	85.25	2001
	SHA-384	384	512 (8 × 64)	1024	80	And, Xor, Or, Rot, Shr, Add (mod 2 ⁶⁴)	192	128	5.12	135.75	2001
	SHA-512	512					256	0 ^[62]	5.06	135.50	2001
	SHA-512/224	224					112	288	≈ SHA-384	≈ SHA-384	2012
	SHA-512/256	256					128	256			
SHA-3	SHA3-224	224	1600 (5 × 5 × 64)	1152	24 ^[63]	And, Xor, Rot, Not	112	448	8.12	154.25	2015
	SHA3-256	256		1088			128	512	8.59	155.50	
	SHA3-384	384		832			192	768	11.06	164.00	
	SHA3-512	512		576			256	1024	15.88	164.00	
	SHAKE128	d (arbitrary)		1344			min(d/2, 128)	256	7.08	155.25	
	SHAKE256	d (arbitrary)		1088			min(d/2, 256)	512	8.59	155.50	

SHA-1 (Secure Hash Algorithm - 1)

SHA-1

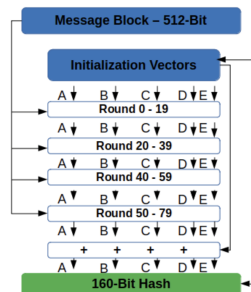
Definition

SHA-1. Is a cryptographic hash function which takes an input and produces a 160-bit (20-byte) hash value known as a message digest – typically rendered as 40 hexadecimal digits. It was designed by the United States National Security Agency in 1995.

It has the following features:

- Deprecated by NIST in 2011.
- Vulnerable to chosen-prefix attacks. In 2017, a collision attack against SHA-1 was successful.
- Still secure for HMAC generation.
- Replaced by SHA-2 and SHA-3.

Operates on 512-bit blocks to generate a 160-bit hash. Was used in early versions of TLS, SSL, Git, and password storage.



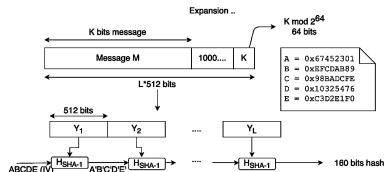
SHA1 hashing process

SHA-1

SHA-1 hashing operates as follows (here a word is expressed in 32 bits):

- 1 Initialize five variables $H_0 = 0x67452301$, $H_1 = 0xefcdab89$, $H_2 = 0x98badcfe$, $H_3 = 0x10325476$, and $H_4 = 0xc3d2e1f0$.

- 2 Format the plaintext by adding padding so that the total length is multiple of 512 bits. Then, break the message into chunks (blocks) of 512 bits.



- 3 For each block, break the block into 16 words $W[i], 0 \leq i \leq 15$:

- 1 Extend those 16 words into 80 words, using:

$$w[i] = (w[i - 3] \oplus w[i - 8] \oplus w[i - 14] \oplus w[i - 16]) \ll 1$$

- 2 Initialize values $A, B, \dots, E$ with $H_0, H_1, \dots, H_4$. Then run the main loop (80 iterations) to compute new values for the variables $A, B, \dots, E$.
- 3 Update H_i : $H_0 += A$, $H_1 += B$, $H_2 += C$, $H_3 += D$, and $H_4 += E$.
- 4 Final hash is: $(H_0 \ll 128) \parallel (H_1 \ll 96) \parallel (H_2 \ll 64) \parallel (H_3 \ll 32) \parallel H_4$

SHA-1

SHA-1 hashing operates as follows (here a word is supposed to be 32 bits):

SHA-1 Main Loop:

```

1  for i from 0 to 79
    if  $0 \leq i \leq 19$  then
         $F = (B \cdot C) + ((\bar{B}) \cdot D)$ 
         $K = 0x5A827999$ 
2  else if  $20 \leq i \leq 39$ 
         $F = B \oplus C \oplus D$ 
         $K = 0x6ED9EBA1$ 
3  else if  $40 \leq i \leq 59$ 
         $F = (B \cdot C) + (B \cdot D) + (C \cdot D)$ 
         $K = 0x8F1BBCDC$ 
    else if  $60 \leq i \leq 79$ 
         $F = B \oplus C \oplus D$ 
         $K = 0xCA62C1D6$ 
    temp =  $(A \ll 5) + F + E + K + w[i]$ 
    E = D; D = C; C = B  $\ll$  30; B = A; A = temp
4  Output(A, B, C, D, E)
  
```

Secure Hash Algorithms (SHA Family)

On Unix-like operating systems, you can use SHA hash function to compute the message integrity code using the `shasum` command:

```
((base) mirak@Karims-MacBook-Pro ~ % echo "ENSIA" >> ensia.exe
((base) mirak@Karims-MacBook-Pro ~ % cat ensia.exe
ENSIA
((base) mirak@Karims-MacBook-Pro ~ % shasum ensia.exe
e7a4b50ff501ba180864109b3630180d59c4377f  ensia.exe
((base) mirak@Karims-MacBook-Pro ~ % shasum -a 256 ensia.exe
cab1d23b677ec107cafd46e5ec572b31d08049041ccd4d50e4330e22c03baa5c  ensia.exe
((base) mirak@Karims-MacBook-Pro ~ % shasum -a 512 ensia.exe
e295c36c0cc4f59dc51f3f7b7f5a92c5897d61e6cfecfece34e069cecef3b7691bdd20abacc
c562a2e58aa2ae01dc59ca95772763a5f3cc4b3655c040902d084  ensia.exe
((base) mirak@Karims-MacBook-Pro ~ % shasum -a 512224 ensia.exe
a9920e3c6aead790e2d1e7dcc39482414534d5cc3f20c9b78dddb3b7  ensia.exe
((base) mirak@Karims-MacBook-Pro ~ %
```

By default, the command invokes SHA-1 hash algorithm. When the output size is specified (e.g., 224, 256, 384, and 512 bits), then SHA-2 family algorithms are used.

Secure Hash Algorithms (SHA Family)

On Unix-like operating systems, you can use `openssl` to invoke SHA-3 hash functions to compute the message integrity code:

```
(base) mirak@Karims-MacBook-Pro ~ % openssl dgst -sha3-256 ensia.exe
SHA3-256(ensia.exe)= 7f8c5758b9ccec5847ae52e9e7feb8871e398eaf73fc17c
44d695538d98dc820
(base) mirak@Karims-MacBook-Pro ~ % openssl dgst -sha3-224 ensia.exe
SHA3-224(ensia.exe)= 8d6c452f22ee0fa80402daa532fcf40b1feed04ba0a2915
f3a4b90f6
(base) mirak@Karims-MacBook-Pro ~ % openssl dgst -sha3-384 ensia.exe
SHA3-384(ensia.exe)= a47bb324fa18ac078d3d3fcdbbf5711cdd7342466b4a6e8
339f73cd253ae98b82d5ebd646c1a1681ca328101cb529182
(base) mirak@Karims-MacBook-Pro ~ % openssl dgst -sha3-512 ensia.exe
SHA3-512(ensia.exe)= a23f2442ca14aef8ba08ca1c8ec9551c58d479b0868eb78
b06357057b804fb8dc38b8d73ad69c626740c01d45e67d144af64ce9037ad561ae5b
3f152d488b31e
```

Attacks on Hash Function

The following are some of the most common attacks against hash functions:

- **Collision attack.** Find/Choose two different inputs m and m' ($m \neq m'$) that produces the same hash value $\mathcal{H}(m) = \mathcal{H}(m')$: (here $\mathcal{K} = \emptyset$)
 - Abuse digital signature (fake message with valid signature).
- **Preimage attack.** Find an input m' that produces a pre-determined hash value v . i.e., $\mathcal{H}(m') = v$: (here $\mathcal{K} = v$)
 - Reverse password hashing (offline password-hash cracking).
 - Craft malware that bypass signature(hash)-based security filters.
- **Second image attack.** Find another input $m' \neq m$ that produces the same hash, i.e., $\mathcal{H}(m) = \mathcal{H}(m')$. (here $\mathcal{K} = \{\mathcal{H}(m), m\}$)
 - Compromise hash-based verified stored files.
- **Length extension attack.** Exploit flaws in hash functions to generate a hash of a longer message given the hash of a shorter message.

☞ The notation $\mathcal{K}$ means the attacker's knowledge of the system.

- **End.**